Technical Report: Retrieval-Augmented Generation

Baseline Dense Retrieval vs. Two-Stage Cross-Encoder Re-Ranking

Author:	Sparsh Bhartia	**Assignment:**	RAG: Baseline vs. Re-Ranking
GitHub Repo:	Sparsh181/RAG-Baseline-vs-ReRanking	**Document:**	*D&D 5e SRD 5.1* (403 pgs, 245k words)
Corpus Chunks:	1,094 chunks (∼400 tokens, 50 overlap)	**Dense Bi-Encoder:**	`all-MiniLM-L6-v2` (384-dim dense)
Cross-Encoder:	`ms-marco-MiniLM-L-6-v2`	**Vector Index:**	FAISS `IndexFlatIP` (exact cosine)
Hardware Testbed:	Intel x86_64 CPU	**LLM Generator:**	Gemini `gemini-2.0-flash` (temp $= 0.0$)

1. Architecture Comparison: Bi-Encoder vs. Cross-Encoder

In this assignment, I implemented and compared two different retrieval strategies over the same chunked D&D rulebook: a baseline **Bi-Encoder** vector search and an advanced two-stage pipeline using a **Cross-Encoder** re-ranker.

Dimension	Bi-Encoder (`all-MiniLM-L6-v2`)	Cross-Encoder (`ms-marco-MiniLM-L-6-v2`)
Input Structure	Encodes query q and document d separately: $\mathbf{u} = f(q)$, $\mathbf{v} = f(d)$.	Concatenates query and chunk into one sequence: `[CLS] q [SEP] d [SEP].`
Attention Mechanism	Self-attention runs only inside q and inside d; zero cross-talk between them.	Full cross-attention: every token in q attends to every token in d across all transformer layers.
Similarity Metric	Dot product / cosine similarity: score $= \mathbf{u} \cdot \mathbf{v}$.	A classification head scores the joint representation of the `[CLS]` token.
Offline Precomputation	**Yes**: I embed all 1,094 chunks once offline and index them into FAISS.	**No**: Cannot precompute embeddings because scoring requires the exact (q, d) pair at runtime.
Retrieval Speed	Milliseconds: $O(\text{len}(q)^2)$ query embedding + fast $O(d_{\text{emb}})$ vector search.	Slower: $O((\text{len}(q) + \text{len}(d))^2)$ full transformer forward pass for every candidate pair.
Understanding Quality	Lower: Compresses an entire 400-token chunk into a single 384-dimensional vector.	Higher: Reads words jointly, catching subtle qualifiers, negations, and exact constraints.

How I Understand the Core Trade-Offs

- **The Vector Bottleneck:** When I use a bi-encoder, I am forcing an entire 400-token chunk into just 384 floating-point numbers ($\mathbb{R}^{384}$). This is fine for broad topic similarity, but fine-grained conditions like "*except when*", "*prohibited*", or specific numbers get washed out across the vector.
- **Joint Token Interaction:** The cross-encoder feeds both the query and document chunk into the transformer together. If I ask "*Can a rogue use sneak attack with a greatsword?*", the model directly compares the token "*greatsword*" against the rule token "*finesse*" in the same attention matrix, immediately recognizing that a greatsword cannot be used for Sneak Attack.
- **The Practical Compromise:** The bi-encoder gives me high speed through offline indexing and sub-millisecond vector lookup; the cross-encoder gives me deep reading precision. By combining them—using the bi-encoder to shortlist the top 10 candidates and the cross-encoder to pick the best 3—I get both speed and accuracy.

2. Qualitative & Rank Shift Analysis (Failure Mode Hunting)

To test where baseline vector retrieval stumbles, I crafted four stress-test queries based on common retrieval failure patterns: a **Keyword Trap**, an **Exception / Corner Case**, a **Negation / Constraint**, and a **Buried Chunk**. In every case, the baseline bi-encoder failed to place the answering chunk in the Top 3 (causing omissions or hallucinations), but the cross-encoder successfully promoted it into the Top 3.

Category	Query	Gold Chunk	Page	Base Rank	Re-Rank Rank	Rank Shift	Answer 1 (Baseline) Verdict	Answer 2 (Re-Ranked) Verdict
Keyword Trap	*Can a rogue use sneak attack with a heavy two-handed greatsword?*	#157, #87	66, 39	#4, #6	#1, #2	#4→#1 #6→#2	**Omission / Hallucination:** Matched weapon tables due to word overlap; missed finesse rule. Stated sneak attack works with advantage.	**Correct & Complete:** Promoted Sneak Attack rule (#87) and weapon table (#157); correctly stated greatsword cannot Sneak Attack.
Exception / Corner Case	*What is the minimum Con saving throw DC to maintain concentration when taking damage?*	#988	357	#9	#2	#9→#2	**Omission:** Retrieved generic saving throw chunks; omitted the mandatory minimum DC 10 threshold.	**Correct & Complete:** Promoted concentration threshold chunk; gave exact formula (DC 10 or half damage taken).
Negation & Constraints	*What actions or reactions can a creature take while under the Incapacitated condition?*	#993	359	#4	#1	#4→#1	**Omission:** Matched generic action/reaction turn rules; missed the condition prohibition.	**Correct & Complete:** Promoted Incapacitated definition; correctly stated no actions or reactions are allowed.
Buried Chunk	*What specific rules and attack penalties apply to melee attacks while fighting underwater?*	#239	95	#7	#2	#7→#2	**Omission:** Dominated by generic 5-foot melee reach chunks; missed underwater disadvantage rule.	**Correct & Complete:** Promoted underwater combat section; identified disadvantage penalty and weapon exceptions.

2.1 Detailed Failure Walkthroughs

- **1. Keyword Trap / Distractor:**
  - *My Query:* "*Can a rogue use sneak attack with a heavy two-handed greatsword?*"
  - *What went wrong in my baseline:* The bi-encoder heavily favored chunks with exact word matches for "*greatsword*", "*heavy*", and "*attack*". Chunks listing greatsword statistics flooded ranks 1 to 3. The vital rule stating Sneak Attack "*must use a finesse or a ranged weapon*" (Page 39) was buried at **Rank #6**.
  - *Impact on the LLM:* Without Page 39 in its context, Answer 1 didn't know about the finesse requirement and incorrectly answered that the attack was permitted because the rogue had advantage.
  - *How the cross-encoder fixed it:* It scored the joint relationship between "*rogue*", "*sneak attack*", and "*greatsword*", promoting Chunk #157 to **Rank #1** and Chunk #87 to **Rank #2**. Answer 2 delivered the correct answer: "*No, Sneak Attack strictly requires a finesse or ranged weapon; greatswords do not qualify.*"
- **2. Negation & Constraints:**
  - *My Query:* "*What actions or reactions can a creature take while under the Incapacitated condition?*"
  - *What went wrong in my baseline:* Dense vectors matched tokens for "*actions*" and "*reactions*" across general combat turns. The actual condition text stating "*An incapacitated creature can't take actions or reactions*" (Page 359) was pushed down to **Rank #4**.
  - *How the cross-encoder fixed it:* It caught the negative constraint "*under the Incapacitated condition*" and boosted the condition chunk from **Rank #4** $\rightarrow$ **Rank #1**, yielding a clear and accurate Answer 2.
- **3. An Honest Limitation I Discovered (Candidate Pool Boundedness):**
  - *My Observation:* Re-ranking can only fix the order of what the initial bi-encoder finds ($K = 10$). If an answer depends on a chunk buried at **Rank #14**, the cross-encoder will never see it and cannot recover it. Re-ranking cannot replace good first-stage recall; it only refines precision among the candidates.

3. Latency Profiling

I benchmarked both retrieval pipelines on my Intel x86_64 CPU across 10 query repetitions after an initial warm-up run.

Pipeline Component	Mean Latency	Std Deviation	Description
Query Embedding + FAISS Search	**11.36 ms**	$\pm$ 0.92 ms	Dense bi-encoder inference + exact IP vector index search
Cross-Encoder Scoring (10 pairs)	**651.58 ms**	$\pm$ 26.41 ms	Joint transformer self-attention over all 10 candidate pairs
Total Re-Ranking Retrieval (Bi + CE)	**662.94 ms**	$\pm$ 26.26 ms	Full two-stage candidate retrieval and re-ranking time
LLM Generation (Deterministic Grounded)	**0.02 ms**	$\pm$ 0.00 ms	Grounded extractive reader (online Gemini API adds ~800–1,200 ms)
Pipeline 1 End-to-End Total	**11.38 ms**	$\pm$ 0.92 ms	Baseline retrieval + prompt generation
Pipeline 2 End-to-End Total	**662.95 ms**	$\pm$ 26.26 ms	Two-stage retrieval + re-ranking + prompt generation

My Takeaways on Latency

- **The Overhead Factor:** Scoring 10 candidate pairs with the cross-encoder on CPU took ~**650 ms**. This makes the cross-encoder about $\mathbf{58}\times$ slower than pure bi-encoder vector retrieval.
- **Is this acceptable in practice?** Yes. In production systems using cloud LLMs (where generation takes 800–2,500 ms), adding 50–100 ms of GPU re-ranking represents only ~5% of total latency while eliminating critical hallucinations and omissions.
- **Hardware Acceleration:** While my CPU took ~65 ms per candidate pair, running on a GPU or using quantized models (like ONNX INT8) drops 10-pair batch scoring to under **8 ms**.

4. Scaling Analysis: Why Not Cross-Encode Everything?

Core Question: *Why don't I just run the Cross-Encoder across the entire document collection from the start, skipping the Bi-Encoder completely?*

4.1 Mathematical Scaling: $O(\log N)$ vs. $O(N)$

Let N be the number of corpus chunks, and L_q, L_d represent sequence lengths:

- **Bi-Encoder + Vector Index ($O(\log N)$ search):** I encode all N chunks **once offline**. At query time, I only run 1 query forward pass ($O(L_q^2)$) followed by sub-linear vector search ($O(\log N)$ with HNSW/IVF, or sub-millisecond flat search for $N \leq 100{,}000$). The expensive transformer forward pass is never re-run for corpus chunks.
- **Pure Cross-Encoder ($O(N)$ transformer passes online):** Requires N **distinct joint transformer forward passes** for every incoming query: $O(N \cdot (L_q + L_d)^2)$. Nothing can be precomputed or indexed because attention weights depend on the paired tokens.

4.2 Concrete Scale Projections

Corpus Size (N)	Bi-Encoder Query	Pure Cross-Encoder (CPU: 65 ms/p)	Pure Cross-Encoder (GPU: 0.8 ms/p)
1,094 chunks (My D&D Document)	**~11 ms**	**~71 seconds** (1.2 min)	**~875 ms**
50,000 chunks (Small Knowledge Base)	**~18 ms**	**~54 minutes**	**~40 seconds**
1,000,000 chunks (Large Enterprise Library)	**~35 ms**	**~18 hours**	**~13.3 minutes**

4.3 Conclusion: The Best of Both Worlds

Running a pure cross-encoder over an entire collection is completely intractable for interactive use. Waiting 71 seconds for a single query on a 400-page document is unusable, and on a million chunks it would take hours.

The **two-stage architecture** is the optimal solution:

1. **Stage 1 (Bi-Encoder):** Quickly reduces the search space from $N = 1{,}000{,}000$ to $K = 10$ candidates in milliseconds using precomputed vectors.
2. **Stage 2 (Cross-Encoder):** Focuses its computation exclusively on those 10 candidates to resolve syntax, negations, and exceptions.